
CHAPTER 1

OBJECT ORIENTED PROGRAMMING

1.1 OOP

Definition 1.1.1 (OOP Object Oriented Programming)

OOP stands for **Object-Oriented Programming**. Object-oriented programming is about *creating objects, which can hold data and functions that work on that data.*

Advantages:

- OOP provides a *clear structure* to programs
- Makes *code easier to maintain, reuse, and debug*
- Helps keep your code DRY (Don't Repeat Yourself)
- Makes it *possible to create full reusable applications with less code and shorter development time*

1.1.1 CLASSES AND OBJECTS

Classes and **objects** are the two main aspects of object-oriented programming.

Definition 1.1.2 (Class)

A **class** *defines what an object should look like, and an object is created based on that class.*

Class	Objects
Fruit	Apple, Banana, Mango
Car	Volvo, Audi, Toyota

Table 1.1:

Observation 1.1.1 (Functions Inside a Class)

When you create an object from a class, it inherits all the variables and functions defined inside that class.

Also, we say that an object *apple* is an instance of the class *fruit*.

1.1.2 PROCEDURAL VS OBJECT-ORIENTED PROGRAMMING

We have two programming focuses:

- **Procedural programming** is about writing functions that operate on data.
- **Object-oriented programming (OOP)** is about creating objects that contain both the data and the functions.

1.2 C++ CLASSES/OBJECTS

C++ is an object-oriented programming language. Everything in C++ is associated with classes and objects, along with its attributes and methods.

Example 1.2.1

For example: in real life, a car is an object. The car has attributes, such as weight and color, and methods, such as drive and brake.

Definition 1.2.1 (Attributes and Methods)

Attributes and **methods** are *basically variables and functions that belongs to the class. These are often referred to as class members.*

A class is a user-defined data type that we can use in our program, and it works as an object constructor, or a blueprint for creating objects.

Example 1.2.2 (Example of a Class)

To create a Class in C++ we do the following:

```
1 class MyClass {           // The class
2     public:               // Access specifier
3         int myNum;         // Attribute (int variable)
4         string myString;   // Attribute (string variable)
5 };
```

Code 1.1: caption

- The **class** keyword is used to create a **class** called **MyClass**.
- The **public** keyword is an **access specifier**, which *specifies that members (attributes and methods) of the class are accessible from outside the class.* You will learn more about access specifiers later.
- Inside the **class**, there is an integer variable **myNum** and a string variable **myString**. When variables are declared within a class, they are called **attributes**.
- At last, *end the class definition with a semicolon ;.*

1.2.1 CREATE AN OBJECT

In C++, an object is created from a **class**. We have already created the **class** named **MyClass**, so now we can use this to create objects.

To create an object of **MyClass**, specify the class name, followed by the object name.

To access the class attributes (**myNum** and **myString**), use the dot syntax (.) on the object:

Example 1.2.3 (Creation of an Object)

Create an object called **myObj** and access the attributes:

```
1 class MyClass {           // The class
2     public:               // Access specifier
3         int myNum;         // Attribute (int variable)
4         string myString;   // Attribute (string variable)
5 };
6
```

```

7  int main() {
8      MyClass myObj; // Create an object of MyClass
9
10     // Access attributes and set values
11     myObj.myNum = 15;
12     myObj.myString = "Some text";
13
14     // Print attribute values
15     cout << myObj.myNum << "\n";
16     cout << myObj.myString;
17     return 0;
18 }

```

Code 1.2: caption

Observation 1.2.1 (Several Objects)

Also, we can create several objects from a same class.

1.3 CLASS METHODS

Definition 1.3.1 (Methods)

Methods are *functions that belongs to the class*.

There are two ways to define functions that belongs to a class:

- Inside class definition
- Outside class definition

1.3.1 DEFINE A METHOD INSIDE THE CLASS

In the following example, we define a function inside the class, and we name it `myMethod`.

Idea 1.3.1 (Note)

You access methods just like you access attributes; by creating an object of the class and using the dot syntax (`.`):

Example 1.3.1 (Method Creation)

Example of a method inside a class:

```

1  class MyClass { // The class
2      public: // Access specifier
3          void myMethod() { // Method/function defined inside the
4              cout << "Hello World!";
5          }
6  };
7
8  int main() {
9      MyClass myObj; // Create an object of MyClass
10     myObj.myMethod(); // Call the method

```

```

11     return 0;
12 }

```

Code 1.3: caption

1.3.2 DEFINE A METHOD OUTSIDE THE CLASS

Sometimes it is *better to declare the method in the class and define it later* (especially in large programs).

This is done by specifying the name of the class, followed the scope resolution `::` operator, followed by the name of the function:

```

1  class MyClass {           // The class
2      public:               // Access specifier
3          void myMethod();   // Method/function declaration
4  };
5
6  // Method/function definition outside the class
7  void MyClass::myMethod() {
8      cout << "Hello World!";
9  }
10
11 int main() {
12     MyClass myObj;         // Create an object of MyClass
13     myObj.myMethod();      // Call the method
14     return 0;
15 }

```

Code 1.4: caption

1.3.3 PARAMETERS

You can also pass values to methods just like regular functions:

Example 1.3.2 (Pass Values Through Methods)

To pass some value to a method we do the following:

```

1  #include <iostream>
2  using namespace std;
3
4  class Car {
5      public:
6          int speed(int maxSpeed);
7  };
8
9  int Car::speed(int maxSpeed) {
10     return maxSpeed;
11 }
12
13 int main() {
14     Car myObj; // Create an object of Car
15     cout << myObj.speed(200); // Call the method with an
        argument
16     return 0;

```

```
17 }
```

Code 1.5: caption

1.4 CONSTRUCTORS

Definition 1.4.1 (Constructor)

A **constructor** is a *special method that is automatically called when an object of a class is created*.

To create a constructor, use the same name as the class, followed by parentheses ():

Creation of a constructor:

```
1 class MyClass {           // The class
2     public:               // Access specifier
3     MyClass() {           // Constructor
4         cout << "Hello World!";
5     }
6 };
7
8 int main() {
9     MyClass myObj;        // Create an object of MyClass (this will
10                            call the constructor)
11     return 0;
12 }
```

Code 1.6: caption

Observation 1.4.1 (Constructor Rules)

When *creating a constructor*, we must have in mind the following rules:

- The constructor *has the same name as the class*.
- It *has no return type* (not even void).
- It is *usually declared public*.
- It is *automatically called when an object is created*.

1.4.1 CONSTRUCTOR WITH PARAMETERS

Definition 1.4.2 (Constructor with Parameters)

Constructors *can also take parameters* (just like regular functions), which *can be useful for setting initial values for attributes*.

The following class has `brand`, `model` and `year` attributes, and a constructor with different parameters. Inside the constructor we set the attributes equal to the constructor parameters (`brand=x`, etc).

When we call the constructor (by creating an object of the class), we pass parameters to the constructor, which will set the value of the corresponding attributes to the same:

```
1 class Car {               // The class
```

```

2   public:           // Access specifier
3   string brand;     // Attribute
4   string model;     // Attribute
5   int year;         // Attribute
6   Car(string x, string y, int z) { // Constructor with
        parameters
7       brand = x;
8       model = y;
9       year = z;
10  }
11 };
12
13 int main() {
14     // Create Car objects and call the constructor with different
        values
15     Car carObj1("BMW", "X5", 1999);
16     Car carObj2("Ford", "Mustang", 1969);
17
18     // Print values
19     cout << carObj1.brand << " " << carObj1.model << " " <<
        carObj1.year << "\n";
20     cout << carObj2.brand << " " << carObj2.model << " " <<
        carObj2.year << "\n";
21     return 0;
22 }

```

Code 1.7: caption

1.4.2 CONSTRUCTOR DEFINED OUTSIDE THE CLASS

You can also define the constructor outside the class using the scope resolution operator ::.

```

1   class Car {       // The class
2   public:           // Access specifier
3       string brand; // Attribute
4       string model; // Attribute
5       int year;     // Attribute
6       Car(string x, string y, int z); // Constructor declaration
7   };
8
9   // Constructor definition outside the class
10  Car::Car(string x, string y, int z) {
11      brand = x;
12      model = y;
13      year = z;
14  }
15
16  int main() {
17      // Create Car objects and call the constructor with different
        values
18      Car carObj1("BMW", "X5", 1999);
19      Car carObj2("Ford", "Mustang", 1969);
20
21      // Print values

```



```

22     cout << carObj1.brand << " " << carObj1.model << " " <<
        carObj1.year << "\n";
23     cout << carObj2.brand << " " << carObj2.model << " " <<
        carObj2.year << "\n";
24     return 0;
25 }

```

Code 1.8: caption

1.4.3 WHY USING CONSTRUCTORS

Constructors **run by themselves when you create an object**. They **set things up so everything is ready right away**.

Think of it like this: When you order a pizza (object), the constructor is the chef who adds the sauce, cheese, and toppings before it gets to you - you don't have to do it yourself!

1.4.4 C++ CONSTRUCTOR OVERLOADING

In C++, you can have more than one constructor in the same class. This is called **constructor overloading**.

Observation 1.4.2

Each constructor *must have a different number or type of parameters, so the compiler knows which one to use when you create an object*.

Idea 1.4.1 (Why Use Constructor Overloading?)

We use constructor overloading to:

- To give flexibility when creating objects
- To set default or custom values
- To reduce repetitive code

Example 1.4.2 (Example with Two Constructors)

This class has two constructors: one without parameters, and one with parameters:

```

1  class Car {
2      public:
3          string brand;
4          string model;
5
6          Car() {
7              brand = "Unknown";
8              model = "Unknown";
9          }
10
11         Car(string b, string m) {
12             brand = b;
13             model = m;
14         }
15     };
16
17     int main() {

```

```

18 Car car1;
19 Car car2("BMW", "X5");
20 Car car3("Ford", "Mustang");
21
22 cout << "Car1: " << car1.brand << " " << car1.model << "\n"
    ;
23 cout << "Car2: " << car2.brand << " " << car2.model << "\n"
    ;
24 cout << "Car3: " << car3.brand << " " << car3.model;
25 return 0;
26 }

```

Code 1.9: caption

The result of executing the following code is this:

```

1 Car1: Unknown Unknown
2 Car2: BMW X5
3 Car3: Ford Mustang

```

Code 1.10: caption

1.5 C++ ACCESS SPECIFIERS

Definition 1.5.1 (Access Specifiers)

Access specifiers control how the members (attributes and methods) of a class can be accessed.

They help protect data and organize code so that only the right parts can be seen or changed.

1.5.1 USING ACCESS SPECIFIERS

The **public** keyword is an access specifier.

Example 1.5.1 (Use of public Keyword)

In the code below, the members are *public* - which means that they can be accessed and modified from outside the code:

```

1 class MyClass { // The class
2     public:      // Access specifier
3     // class members goes here
4 };

```

Code 1.11: caption

In C++, there are three access specifiers:

- **public** - members are accessible from outside the class.
- **private** - members cannot be accessed (or viewed) from outside the class.
- **protected** - members cannot be accessed from outside the class, however, they can be accessed in inherited classes.

1.5.2 PRIVATE

Members declared as **private** cannot be accessed from outside the class.

Example 1.5.2 (Differences Between public and private)

In the following code, we demonstrate the *differences between public and private members*:

```
1 class MyClass {
2     public:      // Public access specifier
3         int x;   // Public attribute
4     private:    // Private access specifier
5         int y;   // Private attribute
6 };
7
8 int main() {
9     MyClass myObj;
10    myObj.x = 25; // Allowed (public)
11    myObj.y = 50; // Not allowed (private)
12    return 0;
13 }
```

Code 1.12: caption

Observation 1.5.1 (Note on private)

It is *possible to access private members of a class using a public method inside the same class*. See the next chapter (Encapsulation) on how to do this.

Idea 1.5.1 (Tip)

It is *considered good practice to declare your class attributes as private (as often as you can)*. This will reduce the possibility of yourself (or others) to mess up the code. This is also the main ingredient of the **Encapsulation** concept, which you will learn more about in the next chapter.

By default, all members of a class are **private** if you don't specify an access specifier:

```
1 class MyClass {
2     int x;   // Private attribute
3     int y;   // Private attribute
4 };
```

Code 1.13: caption

1.5.3 PROTECTED

Members declared as **protected** cannot be accessed from outside the class, but they can be accessed in child classes.

1.6 C++ ENCAPSULATION

Definition 1.6.1 (Encapsulation)

The meaning of **Encapsulation**, is to make sure that *sensitive data is hidden from users*.

To achieve this, you must *declare class variables/attributes as private (cannot be accessed from outside the class)*.

If you want others to read or modify the value of a private member, you *can provide public get and set methods*.

Example 1.6.1 (Employee's Salary)

Think of an employee's salary:

- The **salary is private** - the employee can't change it directly
- Only their manager can update it or share it when appropriate

Encapsulation works the same way. The **data is hidden, and only trusted methods can access or modify it.**

1.6.1 ACCESS PRIVATE MEMBERS

To access a private attribute, use public `get` and `set` methods:

```
1 #include <iostream>
2 using namespace std;
3
4 class Employee {
5     private:
6         // Private attribute
7         int salary;
8
9     public:
10        // Setter
11        void setSalary(int s) {
12            salary = s;
13        }
14        // Getter
15        int getSalary() {
16            return salary;
17        }
18 };
19
20 int main() {
21     Employee myObj;
22     myObj.setSalary(50000);
23     cout << myObj.getSalary();
24     return 0;
25 }
```

Code 1.14: caption

Observation 1.6.1 (Explanation of Encapsulation)

Example explained

- `salary` is `private` - it cannot be accessed directly
- `setSalary()` sets the value
- `getSalary()` returns the value

We use `myObj.setSalary(50000)` to assign a value, and `myObj.getSalary()` to print it.

Observation 1.6.2 (Why Encapsulation?)

We use encapsulation in order to:

- It is considered good practice to declare your class attributes as private (as often as you can). Encapsulation ensures better control of your data, because you (or others) can change one part of the code without affecting other parts
- Increased security of data

1.7 C++ THE FRIEND KEYWORD

Normally, private members of a class can only be accessed using public methods like getters and setters. But in some cases, you *can use a special function called a friend function to access them directly*.

A **friend** function is *not a member of the class, but it is allowed to access the class's private data* as in the following example:

```
1 class Employee {
2     private:
3         int salary;
4
5     public:
6         Employee(int s) {
7             salary = s;
8         }
9
10        // Declare friend function
11        friend void displaySalary(Employee emp);
12 };
13
14 void displaySalary(Employee emp) {
15     cout << "Salary: " << emp.salary;
16 }
17
18 int main() {
19     Employee myEmp(50000);
20     displaySalary(myEmp);
21     return 0;
22 }
```

Code 1.15: caption

- The **friend** function `displaySalary()` is declared inside the `Employee` class but defined outside of it.
- Even though `displaySalary()` is not a member of the class, it can still access the private member `salary`.
- In the `main()` function, we create an `Employee` object and call the **friend** function to print its salary.

1.8 INHERITANCE

Definition 1.8.1 (Inheritance)

Inheritance allows one class to reuse attributes and methods from another class. It helps you write cleaner, more efficient code by avoiding duplication.

We group the *inheritance concept* into two categories:

- **derived class (child)** - the *class that inherits from another class*
- **base class (parent)** - the *class being inherited from*

To inherit from a class, use the `:` symbol.

Example 1.8.1 (Example of Inheritance)

In the code below, the `Car` class (child) inherits the attributes and methods from the `Vehicle` class (parent):

```

1 // Base class
2 class Vehicle {
3     public:
4         string brand = "Ford";
5         void honk() {
6             cout << "Tuut, tuut! \n" ;
7         }
8 };
9
10 // Derived class
11 class Car: public Vehicle {
12     public:
13         string model = "Mustang";
14 };
15
16 int main() {
17     Car myCar;
18     myCar.honk();
19     cout << myCar.brand + " " + myCar.model;
20     return 0;
21 }
```

Code 1.16: caption

1.8.1 MULTILEVEL INHERITANCE

A class can also be derived from one class, which is already derived from another class.

Example 1.8.2 (Example of Multilevel Inheritance)

In the following example, `MyGrandChild` is derived from class `MyChild` (which is derived from `MyClass`):

```

1 // Base class (parent)
2 class MyClass {
3     public:
4         void myFunction() {
5             cout << "Some content in parent class." ;
6         }
7 };
8
9 // Derived class (child)
10 class MyChild: public MyClass {
11 }
```

```

12
13 // Derived class (grandchild)
14 class MyGrandChild: public MyChild {
15 };
16
17 int main() {
18     MyGrandChild myObj;
19     myObj.myFunction();
20     return 0;
21 }

```

Code 1.17: caption

1.8.2 C++ MULTIPLE INHERITANCE

A class can also be derived from more than one base class, using a comma-separated list:

```

1 // Base class
2 class MyClass {
3     public:
4     void myFunction() {
5         cout << "Some content in parent class." ;
6     }
7 };
8
9 // Another base class
10 class MyOtherClass {
11     public:
12     void myOtherFunction() {
13         cout << "Some content in another class." ;
14     }
15 };
16
17 // Derived class
18 class MyChildClass: public MyClass, public MyOtherClass {
19 };
20
21 int main() {
22     MyChildClass myObj;
23     myObj.myFunction();
24     myObj.myOtherFunction();
25     return 0;
26 }

```

Code 1.18: caption

1.8.3 C++ INHERITANCE ACCESS

You learned from the Access Specifiers chapter that there are three specifiers available in C++.

Until now, we have only used **public** (members of a class are accessible from outside the class) and **private** (members can only be accessed within the class).

The third specifier, **protected**, is similar to private, but it can also be accessed in the inherited class:

```

1 // Base class

```

```

2 class Employee {
3     protected: // Protected access specifier
4         int salary;
5 };
6
7 // Derived class
8 class Programmer: public Employee {
9     public:
10        int bonus;
11        void setSalary(int s) {
12            salary = s;
13        }
14        int getSalary() {
15            return salary;
16        }
17 };
18
19 int main() {
20     Programmer myObj;
21     myObj.setSalary(50000);
22     myObj.bonus = 15000;
23     cout << "Salary: " << myObj.getSalary() << "\n";
24     cout << "Bonus: " << myObj.bonus << "\n";
25     return 0;
26 }

```

Code 1.19: caption

1.8.4 POLYMORPHISM

Polymorphism means *many forms*, and it occurs when we have many classes that are related to each other by inheritance.

Like we specified in the previous chapter; *Inheritance lets us inherit attributes and methods from another class. Polymorphism uses those methods to perform different tasks.* This allows us to *perform a single action in different ways*.

For example, imagine a base class **Animal** with a method called **makeSound()**. Derived classes of **Animals** could be **Pigs**, **Cats**, **Dogs**, **Birds**, etc... Every animal can *make a sound*, but each one sounds different:

- **Pig**: wee wee
- **Dog**: bow wow
- **Bird**: tweet tweet

This is polymorphism - the same action (making a sound) behaves differently for each animal:

```

1 // Base class
2 class Animal {
3     public:
4         void animalSound() {
5             cout << "The animal makes a sound \n";
6         }
7 };
8
9 // Derived class

```



```

10 class Pig : public Animal {
11     public:
12         void animalSound() {
13             cout << "The pig says: wee wee \n";
14         }
15 };
16
17 // Derived class
18 class Dog : public Animal {
19     public:
20         void animalSound() {
21             cout << "The dog says: bow wow \n";
22         }
23 };

```

Code 1.20: caption

Now we can create Pig and Dog objects and override the `animalSound()` method:

```

1 // Base class
2 class Animal {
3     public:
4         void animalSound() {
5             cout << "The animal makes a sound \n";
6         }
7 };
8
9 // Derived class
10 class Pig : public Animal {
11     public:
12         void animalSound() {
13             cout << "The pig says: wee wee \n";
14         }
15 };
16
17 // Derived class
18 class Dog : public Animal {
19     public:
20         void animalSound() {
21             cout << "The dog says: bow wow \n";
22         }
23 };
24
25 int main() {
26     Animal myAnimal;
27     Pig myPig;
28     Dog myDog;
29
30     myAnimal.animalSound();
31     myPig.animalSound();
32     myDog.animalSound();
33     return 0;
34 }

```

1.8.5 C++ VIRTUAL FUNCTIONS

Definition 1.8.2 (Virtual Function)

A **virtual function** is a *member function in the base class that can be overridden in derived classes*.

Virtual functions are a key part of polymorphism in C++. They let different objects respond differently to the same function call.

Why Use Virtual Functions? Without virtual, C++ decides which function to call based on the pointer type, not the actual object type.

With virtual, it checks the actual object the pointer is pointing to.

Or to put it even more simply:

Without virtual: the base function runs, even if the object is from a child class. With virtual: the child's version runs, like you expect. Without Virtual Function

Observation 1.8.1 (Explanation of Virtual)

In C++, when you write `Shape* s = new Circle();`, you are storing a pointer to a `Circle` object in a variable typed as `Shape*`.

Without virtual functions, C++ decides which method to call based only on the static type of the pointer (`Shape*`), so calling `s->draw()` will execute `Shape::draw()` even though the object is actually a `Circle`. This is because non-virtual methods use compile-time (static) binding. Virtual functions exist to change this behavior: when a method is declared virtual, C++ uses runtime (dynamic) binding and calls the method corresponding to the object's real type, so `s->draw()` correctly calls `Circle::draw()`. In short, virtual functions allow derived objects to behave according to their actual type when accessed through base-class pointers or references.